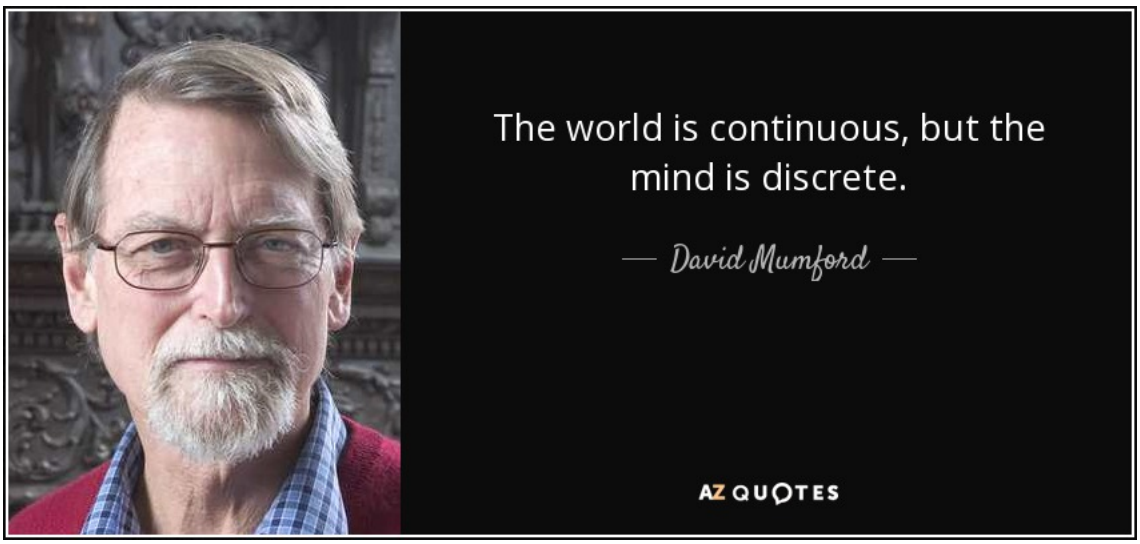
Logic-III

Lecture Credo

> *"I see, I forget.*
> *I hear, I remember.*
> *I do, I understand."*
>
> — Confucius

In this lecture we will learn about:

- Normal Forms
- Application of logic - Boolean Circuit and Circuit Minimization.



Student Name: _____

Student ID: _____

Semester: _____

Welcome, to sheet-3 of logic, Discrete mathematics(M4) Course.

We have revised and learned.

Self Reflection

a) Do you understand Predicate vs Propositions?

☐ Yes ☐ No

Ex. *If **No**, What's the doubt?*

b) Do you understand Quantifiers?

☐ Yes ☐ No

c) Do you know the difference between $\forall$ and $\exists$?

☐ Yes ☐ No

d) Do you understand Nested Quantifiers?

☐ Yes ☐ No

e) Do you understand the relationship established between Logic and Coding so far?

☐ Yes ☐ No

Remember our one of the anchors of the course of **"Game of Mathematics"**. So far it seems we are learning the Game of Logic, and in our Game we have following structure.

Objective

Rules

Definitions

(Fill this up)

We have defined the terms:

- Propositions
- Predicate
- Quantifiers

Some Rules:

- Order of Quantifiers
- Universe of discourse of variables in predicates
- Precedence of logical operators

We can discover more patterns and turn them into rules of the **Game of Logic**, a game we will play throughout our lives.

In Sheet–1, we learned about **logical equivalences**. Once a logical equivalence is established, we can record it as a rule of the game—also known as a **law of logic**. One such rule that we discovered is **De Morgan's Law**.

Let's try to fill it!

T denotes the compound proposition that is always true and **F** denotes the compound proposition that is always false.

Logical Equivalences	
Equivalence	**Name**
$p \wedge \mathbf{T} \equiv$ _____ $p \vee \mathbf{F} \equiv$ _____	
$p \vee \mathbf{T} \equiv$ _____ $p \wedge \mathbf{F} \equiv$ _____	
$p \vee p \equiv$ _____ $p \wedge p \equiv$ _____	
$\neg(\neg p) \equiv$ _____	
$p \vee q \equiv$ _____ $p \wedge q \equiv$ _____	
$(p \vee q) \vee r \equiv$ _____ $(p \wedge q) \wedge r \equiv$ _____	
$p \vee (q \wedge r) \equiv$ _____ $p \wedge (q \vee r) \equiv$ _____	
$\neg(p \wedge q) \equiv$ _____ $\neg(p \vee q) \equiv$ _____	
$p \vee (p \wedge q) \equiv$ _____ $p \wedge (p \vee q) \equiv$ _____	
$p \vee \neg p \equiv$ _____ $p \wedge \neg p \equiv$ _____	

Logical Equivalences Involving Conditional Statements
$p \to q \equiv$ _____
$p \to q \equiv$ _____
$p \vee q \equiv$ _____
$p \wedge q \equiv$ _____
$\neg(p \to q) \equiv$ _____

Logical Equivalences Involving Biconditional Statements
$p \leftrightarrow q \equiv$ _____
$p \leftrightarrow q \equiv$ _____
$p \leftrightarrow q \equiv$ _____
$\neg(p \leftrightarrow q) \equiv$ _____

1 Normal Forms

Introduction

Now that we have established the core concepts of propositional logic, we will turn to some bigger and more applied questions. We will spend the rest of this lesson exploring two specific ways of representing propositions: ***Circuits***, the wire and connections from which physical computers are built and ***normal forms***, in which the structure of propositions is restricted in a particular way.

Let's learn about **Normal Forms** which will use the laws which we have established just now!

We have considered propositions, propositional variables and the connectives $\wedge, \vee, \neg, \to, \leftrightarrow$.

New Notation Alert!

A well formed formula (wff) of propositional logic, also called *propositional form*, or simply formula, is a string consisting of propositional variables, connectives and parentheses used in the proper manner. For example

$$(p \vee q) \wedge (\neg p \vee q \vee r) \wedge (\neg s \vee q)$$

is a wff with propositional variables p, q, r, s.

Ex. *Write one more example.*

a) Can you write the expression of disjunction in terms of p, q, r, s?

Ex. *Write your answer below*

Sometimes it is convenient to use the word "product" for _____ and the word "sum" for _____.

A product of variables and their negations in a formula is called an ***elementary product***.

Ex. *Write some examples.*

Now, based on above understanding of **elementary product**, Now can you define **elementary sum** with some examples?

Ex. *Write your answer below.*

A variable or the negation of a variable is called a ***literal***.
Hence an elementary sum is a _____ and an elementary product is a _____.

Observation

It is easy to observe that

- A necessary and sufficient condition for an elementary product to be identically false (a contradiction) is that it contains at least one pair of literals in which one is the negation of the other.

  If $p \wedge \neg p \wedge \cdots$ is the formula, this reduces to $\mathbf{F}$ and is equal to $\mathbf{F}$.

- A necessary and sufficient condition for an elementary sum to be identically true (a tautology) is that it contains at least one pair of literals in which one is the negation of the other.

  If $p \vee \neg p \vee \cdots$ is a formula, this reduces to $\mathbf{T}$ and is equal to $\mathbf{T}$.

Disjunctive normal form (DNF)

A formula which is equivalent to a given formula and consists of a sum of elementary products is called a ***disjunctive normal form (DNF)***.

To bring any formula to disjunctive normal form
Step 1: First ***replace*** $\rightarrow$ and $\leftrightarrow$ ***using*** $\wedge$ and $\vee$ and $\neg$.
Step 2: Then use ***De Morgan's law*** whenever necessary.

Example: Obtain disjunctive normal forms of

1. $(p \rightarrow q) \wedge \neg q$

Ex. *Write your answer below*

2. $\neg(p \wedge q) \leftrightarrow (p \vee q)$

Ex. *Write your answer below*

This is an equivalent disjunctive normal form. We can see that a given formula is identically false (a contradiction), if every elementary product appearing in its disjunctive normal form is identically false.

Conjunctive normal form(CNF)

A formula which is equivalent to a given formula and consists of a product of elementary sums is called a ***conjunctive normal form*** of the given formula.

We can bring any formula to conjunctive normal form. It should be noted that conjunctive normal form is not unique.

Example: Bring to conjunctive normal form:

1. $(p \to q) \wedge \neg q$
2. $\neg(p \wedge q) \leftrightarrow (p \vee q)$

1. $(p \to q) \wedge \neg q$ is equivalent to

_____

2. $\neg(p \wedge q) \leftrightarrow (p \vee q)$

This is equal to

_____

i.e.,

_____

_____

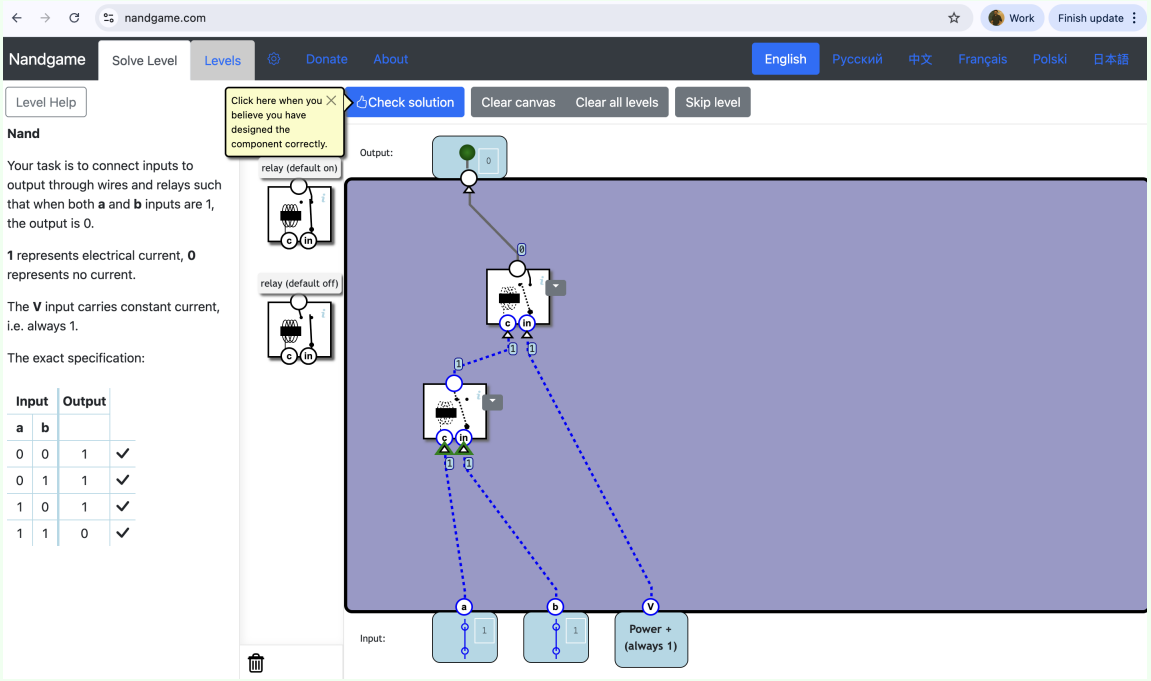
_____

"Let's apply all the gained knowledge of propositional logic in designing boolean circuits which are essential design of computing systems which you are using everyday."

Let's introduce one of the challenge problem which you can replace your contest.

2 Challenge Problem

Nand Game



Description

Challenge is to complete all the levels by yourself (try to do it, the website is about the Game, in which you have to complete all the levels and at least you will have a working computer built from scratch using logic!!
See, the people in the world have implemented the idea of Game, literally they have created GAME OF NAND GATES to learn how computers are made.
Challenge is to complete all levels and at last walk through with explanations of all levels on board and markers.

Best of Luck!

Let us help you understand Circuits of Computers you have been using all along.

3 Introduction

Logic(Abstract Math)

Identify the logical operations.
a)

p	q	$p?q$
T	T	T
T	F	T
F	T	T
F	F	F

Ex. *Write your answer below*

b)

p	q	$p?q$
T	T	T
T	F	F
F	T	F
F	F	F

Ex. *Write your answer below*

Circuits(Real world Application)

Now if we take

$$\text{True} \equiv 1$$
$$\text{False} \equiv 0$$

Rewrite the Table
a)

p	q	

b)

p	q	

We can treat

OR as +
AND as (.)

OR as _____
AND as _____

Exercise

a. Find the value of

$$1 \cdot 0 + \overline{(0+1)}$$

where the bar denotes the **complement** (logical NOT), i.e.,

$$\overline{1} = 0 \quad \text{and} \quad \overline{0} = 1.$$

Ex. *Write your answer below*

b. Translate the above expression into an equivalent **logical expression** using logical connectives $(\wedge, \vee, \neg)$ based on the rules discussed earlier.

Ex. *Write your answer below*

4 Boolean Expression and Boolean Function

Let $B = \{0, 1\}$.
Then

$$B^n = \{(x_1, x_2, \ldots, x_n) \mid x_i \in B,\ 1 \leq i \leq n\}.$$

Here B^n is the set of all possible n-tuples of 0's and 1's.

Quick Question: How many elements will be there in B^n.

Ex. *Write your answer below*

The variable x is called a **Boolean variable** if it assumes values from B, i.e., 0 or 1.

A function from B^n to B is called a **Boolean function of degree** n.

$$f : \underline{\qquad\qquad} \longrightarrow \underline{\qquad\qquad}$$

Follow-up Exercise:

Given a Boolean function of degree 3, i.e.,

$$f : B^3 \to B$$
$$f(x, y, z) = xy + \overline{z}.$$

Find the values of $f(x, y, z)$ by filling in the following table.

x	y	z	xy	$\overline{z}$	$f(x, y, z) = xy + \overline{z}$

In the **Game of Boolean Algebra**, the rules of functions that we have learned earlier are applicable here as well.

a) Two Boolean functions F and G of n variables are said to be **equal** if and only if

$$F(b_1, b_2, \ldots, b_n) = G(b_1, b_2, \ldots, b_n),$$

for all $(b_1, b_2, \ldots, b_n) \in B^n$.

b) Two different Boolean expressions that represent the same Boolean function are called **equivalent**.

Example:

$$xy, \quad xy + 0, \quad xy \cdot 1$$

are equivalent Boolean expressions.

Ex. *Write one more example below*

Similarly one can define:

a) Sum of Boolean Functions

Ex. *Write your answer below*

b) Product of Boolean Functions

Ex. *Write your answer below*

c) Complement of Boolean Function

Ex. *Write your answer below*

5 Representation of Boolean Functions

We will learn about two important problems of Boolean algebra.

First Problem: Given the values of a Boolean function, how can a Boolean expression that represents the function be found?

Try doing it first yourself, as if you are discovering the pattern.

Exercise

Given the table below, find the Boolean functions

$$F(x, y, z) \quad \text{and} \quad G(x, y, z)$$

which map elements from B^3 to B.

x	y	z	F	G
1	1	1	0	0
1	1	0	0	1
1	0	1	1	0
1	0	0	0	0
0	1	1	0	0
0	1	0	0	0
0	0	1	1	1
0	0	0	0	0

Task: Using the table above, determine Boolean expressions for $F(x, y, z)$ and $G(x, y, z)$.

Ex. *Write your answer below*

The solution to this problem is that **every Boolean function can be represented using three Boolean operations**:

$$+, \ \cdot, \ \text{and} \ ^{-}$$

that is, **sum**, **product**, and **complement**.

Terminology Alert

Literal and Minterm

A *literal* is a Boolean variable or its complement. A *minterm* of the Boolean variables $x_1, x_2, \ldots, x_n$ is a Boolean product $y_1 y_2 \cdots y_n$, where $y_i = x_i$ or $y_i = \overline{x_i}$. Hence, a minterm is a product of n literals, with one literal for each variable.

A minterm has the value 1 for one and only one combination of values of its variables. More precisely, the minterm $y_1 y_2 \cdots y_n$ is 1 if and only if each y_i is 1, and this occurs if and only if $x_i = 1$ when $y_i = x_i$ and $x_i = 0$ when $y_i = \overline{x_i}$.

a) Find a minterm that equals 1 if $x_1 = x_3 = 0$ and $x_2 = x_4 = x_5 = 1$, and equals 0 otherwise.

Ex. *Write your answer below*

b) Find the sum-of-products expansion for the function

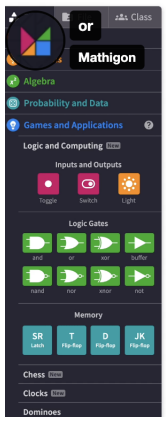
$$F(x, y, z) = (x + y)\,\overline{z}.$$

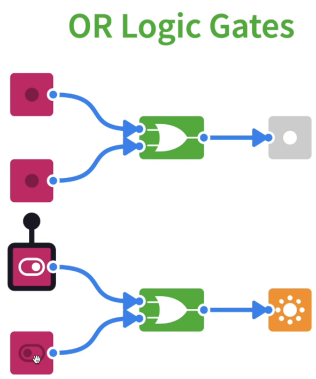
Ex. *Write your answer below*

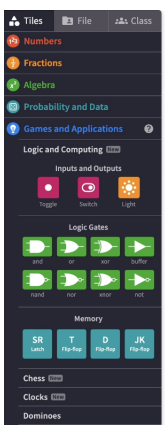
6 Logic Gates

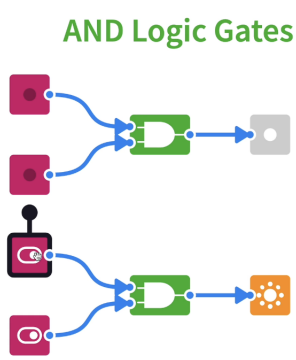
Introduction

Boolean algebra is used to model the circuitry of electronic devices. Each input and each output of such a device can be thought of as a member of the set $\{0, 1\}$. A computer, or other electronic device, is made up of a number of circuits. Each circuit can be designed using the rules of Boolean algebra. The basic elements of circuits are called **gates**.
The circuits give output that depends only on the input, and not on the current state of the circuit. In other words, these circuits have no memory capabilities. Such circuits are called **combinational circuits** or **gating networks**.









Self Reflection

Did you find it interesting?

☐ Yes ☐ No

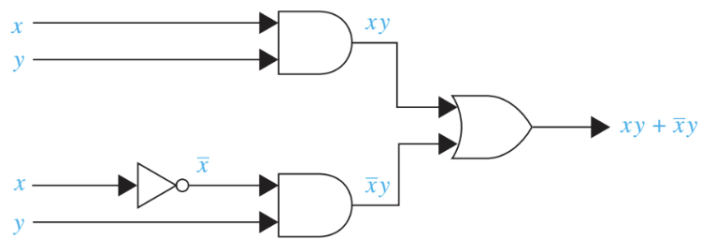
If ***Yes***, open the link below to explore more **Logic Gates**, :
https://polypad.amplify.com/lesson/logic

6.1 Combinations of Gates

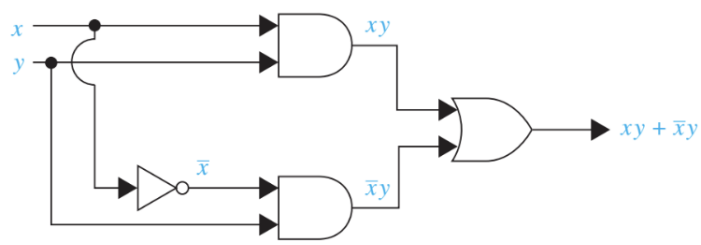
Combinational circuits can be constructed using a combination of inverters, OR gates, and AND gates. When combinations of circuits are formed, some gates may share inputs.

This is shown in one of two ways in depictions of circuits.

Method 1: To use branchings that indicate all the gates that use a given input.



Method 2: To indicate this input separately for each gate.



Example: Construct circuits that produce the following outputs:

$$(x + y)\,\overline{x}$$

Ex. *Construct Circuit below*

6.2 Examples of Circuits

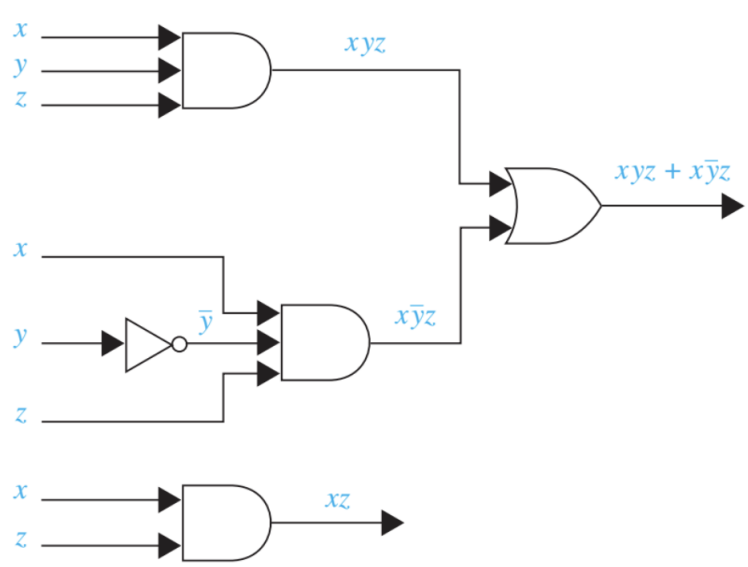
a) A committee of three individuals decides issues for an organization. Each individual votes either yes or no for each proposal that arises. A proposal is passed if it receives at least two yes votes. Design a circuit that determines whether a proposal passes.

Ex. *Construct Circuit and Write your answer below*

b) Sometimes light fixtures are controlled by more than one switch. Circuits need to be designed so that flipping any one of the switches for the fixture turns the light on when it is off and turns the light off when it is on. Design circuits that accomplish this when there are two switches and when there are three switches.

Ex. *Construct Circuit and Write your answer below*

c) Verify both circuits are equivalent.



Ex. *Write your answer below*

6.3 Problem: Is the goal to minimize the circuit?

6.3.1 History Fact

Until the early 1960s logic gates were individual components. To reduce costs it was important to use the fewest gates to produce a desired output. However, in the mid-1960s, integrated circuit technology was developed that made it possible to combine gates on a single chip. Even though it is now possible to build increasingly complex integrated circuits on chips at low cost, minimization of Boolean functions remains important.

Practical Need

Reducing the number of gates on a chip can lead to a more reliable circuit and can reduce the cost to produce the chip. Also, minimization makes it possible to fit more circuits on the same chip. Furthermore, minimization reduces the number of inputs to gates in a circuit. This reduces the time used by a circuit to compute its output. Moreover, the number of inputs to a gate may be limited because of the particular technology used to build logic gates.

The procedure we will introduce, known as Karnaugh maps (or K-maps), was designed in the 1950's to help minimize circuits by hand.

Karnaugh Maps

Introduction to Karnaugh Maps

- To reduce the number of terms in a Boolean expression representing a circuit, it is necessary to find terms that can be combined.
- There is a graphical method, called a **Karnaugh map** or **K-map**, for finding terms to combine for Boolean functions involving a relatively small number of variables.
- This method is usually applied only when the Boolean function involves six or fewer variables.
- K-maps give a visual method for simplifying sum-of-products expansions; they are not suited for mechanizing this process.
- K-maps are first used to simplify Boolean functions in two variables.
- They are then used to minimize Boolean functions in three variables and then in four variables.
- Concepts are later introduced to extend K-maps to minimize Boolean functions in more than four variables.

- There are four possible minterms in the sum-of-products expansion of a Boolean function in two variables x and y.
- A K-map for a Boolean function in two variables consists of four cells.
- A 1 is placed in a cell if the minterm represented by that cell is present in the expansion.
- Cells are said to be **adjacent** if the minterms they represent differ in exactly one literal.
- For example, the cell representing $\overline{x}y$ is adjacent to the cells representing xy and $\overline{x}\,\overline{y}$.
- The four cells and the terms that they represent are shown in Figure.

	y	$\overline{y}$
x	xy	$x\overline{y}$
$\overline{x}$	$\overline{x}y$	$\overline{x}\,\overline{y}$

Solved Example

Find the K-maps for

$$\text{(a) } xy + \overline{x}y, \quad \text{(b) } x\overline{y} + \overline{x}y, \quad \text{and (c) } x\overline{y} + \overline{x}y + \overline{x}\,\overline{y}.$$

Solution: We include a 1 in a cell when the minterm represented by this cell is present in the sum-of-products expansion. The three K-maps are shown below.

	y	$\overline{y}$
x		
$\overline{x}$		

(a)

	y	$\overline{y}$
x		
$\overline{x}$		

(b)

	y	$\overline{y}$
x		
$\overline{x}$		

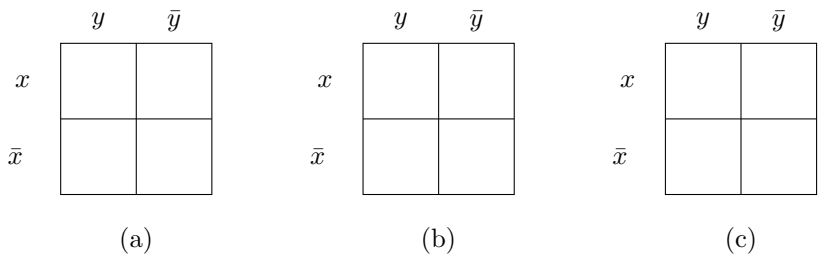
(c)

We can identify minterms that can be combined from the K-map. Whenever there are 1s in two adjacent cells in the K-map, the minterms represented by these cells can be combined into a product involving just one of the variables. For instance, $x\overline{y}$ and $\overline{x}\,\overline{y}$ are represented by adjacent cells and can be combined into $\overline{y}$, because

$$x\overline{y} + \overline{x}\,\overline{y} = (x + \overline{x})\overline{y} = \overline{y}.$$

Moreover, if 1s are in all four cells, the four minterms can be combined into one term, namely the Boolean expression 1 that involves none of the variables. We circle blocks of cells in the K-map that represent minterms that can be combined and then find the corresponding sum of products. The goal is to identify the largest possible blocks and to cover all the 1s with the fewest blocks, always using the largest possible blocks first.

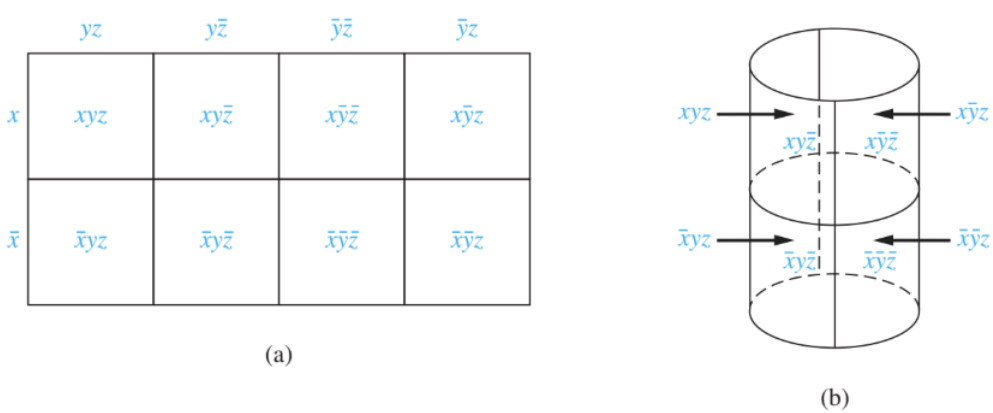
Task: Simplify the sum-of-products expansions given in solved Example.



Ex. *Write your answer below*

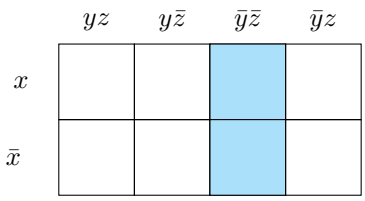
K-map in three variables

A K-map in three variables is a rectangle divided into eight cells. The cells represent the eight possible minterms in three variables. Two cells are said to be adjacent if the minterms that they represent differ in exactly one literal. One of the ways to form a K-map in three variables is shown in Figure (a).
This K-map can be thought of as lying on a cylinder, as shown in Figure (b). On the cylinder, two cells have a common border if and only if they are adjacent.

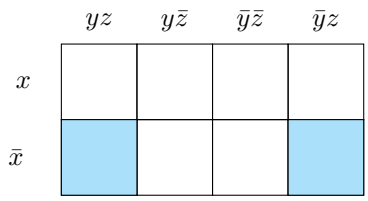


Blocks in K-maps in Three Variables

Try to complete it!

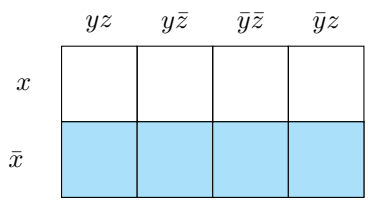


(a)

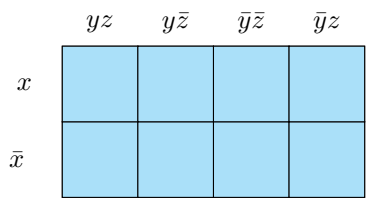


(b)

(c)



(d)



(e)

To simplify a sum-of-products expansion in three variables, we use the K-map to identify blocks of minterms that can be combined. Blocks of two adjacent cells represent pairs of minterms that can be combined into a product of two literals; 2×2 and 4×1 blocks of cells represent minterms that can be combined into a single literal; and the block of all eight cells represents a product of no literals, namely, the function 1.

> The product of literals corresponding to a block of all 1s in the K-map is called an **implicant** of the function being minimized. It is called a **prime implicant** if this block of 1s is not contained in a larger block of 1s representing the product of fewer literals than in this product.
>
> The goal is to identify the largest possible blocks in the map and cover all the 1s in the map with the least number of blocks, using the largest blocks first. The largest possible blocks are always chosen, but we must always choose a block if it is the only block of 1s covering a 1 in the K-map. Such a block represents an **essential prime implicant**. By covering all the 1s in the map with blocks corresponding to prime implicants we can express the sum of products as a sum of prime implicants. Note that there may be more than one way to cover all the 1s using the least number of blocks.

Follow up Exercise

Use K-maps to minimize these sum-of-products expansions.

(a) $xy\bar{z} + x\bar{y}\bar{z} + \bar{x}yz + \bar{x}\bar{y}\bar{z}$

(b) $xy\bar{z} + x\bar{y}\bar{z} + \bar{x}yz + \bar{x}\bar{y}z + \bar{x}\bar{y}\bar{z}$

(c) $xyz + xy\bar{z} + x\bar{y}z + x\bar{y}\bar{z} + \bar{x}yz + \bar{x}\bar{y}z + \bar{x}\bar{y}\bar{z}$

(d) $xy\bar{z} + x\bar{y}\bar{z} + \bar{x}\bar{y}z + \bar{x}\bar{y}\bar{z}$

Try to fill it!

	yz	$y\bar{z}$	$\bar{y}\bar{z}$	$\bar{y}z$
x				
$\bar{x}$				

(a)

	yz	$y\bar{z}$	$\bar{y}\bar{z}$	$\bar{y}z$
x				
$\bar{x}$				

(b)

	yz	$y\bar{z}$	$\bar{y}\bar{z}$	$\bar{y}z$
x				
$\bar{x}$				

(c)

	yz	$y\bar{z}$	$\bar{y}\bar{z}$	$\bar{y}z$
x				
$\bar{x}$				

(d)

Ex. *Write your answer below*